EECS 482
Introduction to operating systems

Programming with monitors

Peter Chen
University of Michigan

Thread-safe queue with condition variables

```
mutex queueMutex;
cv queueCV;
enqueue()
    queueMutex.lock()
    add new element to tail of queue

    queueCV.signal()

    queueMutex.unlock()
dequeue()
    queueMutex.lock()
while if (queue is empty) {

        queueCV.wait(queueMutex)

    }
    remove item from queue
    queueMutex.unlock()
    return removed item
```

atomic

```
queueMutex.unlock()
add myself to waiting set
go to sleep
queueMutex.lock()
```

But this is still broken!
Always use while around wait!

Who is responsible for ensuring condition is met?

- When waiter is woken, it doesn't hold the lock
  - So, it must re-check the condition it was waiting for
  - When could it avoid re-checking?

- Most (all?) languages and operating systems use such condition variables
  - Waiter is responsible for ensuring condition is met
  - Signaller is responsible for telling waiter to re-check condition

Thread-safe queue with condition variables

```
mutex queueMutex;
cv queueCV;
enqueue()
    queueMutex.lock()
    add new element to tail of queue
    queueCV.signal()
    queueMutex.unlock()

dequeue()
    queueMutex.lock()
    while (queue is empty) {
        queueCV.wait(queueMutex)
    }
    remove item from queue
    queueMutex.unlock()
    return removed item
```

Compare busy waiting and cv.wait

```
lock
. . .
while (queue is empty) {
    unlock
    lock
}
. . .
unlock
```

```
lock
. . .
while (queue is empty) {
    cv.wait
}
. . .
unlock
```

unlock
add thread to waiting set
go to sleep
lock

Monitors

- Two mechanisms: one for each type of synchronization
  - Locks for mutual exclusion
  - Condition variables for ordering constraints
- A monitor = a lock + the condition variables associated with that lock
- Mesa monitors: waiter bears sole responsibility for ensuring condition is met before continuing
  - Always use while(...) {wait}

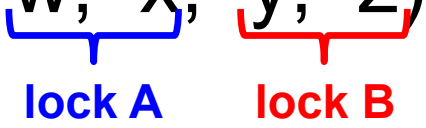
Programming with monitors: overall design

- Think about each thread independently
  - Each thread should try to make as much forward progress as possible (be greedy)
  - A thread should wait only when it is unable to proceed

- List the shared data needed for the problem (e.g., w, x, y, z)

Programming with monitors: mutual exclusion

- List the shared data needed for the problem (e.g., w, x, y, z)



- Assign a lock per group of related, shared data
  - lock…unlock around accesses to shared data
  - Coarse-grained versus fine-grained locking

Programming with monitors: ordering

- When is a thread unable to make progress?
- Use one condition variable for each before-after condition
  - Condition variable belongs to the lock
    that protects the shared data
    that is used to evaluate the condition
- Waiting thread uses while(condition) {wait}
- Signalling thread calls signal() or broadcast() when it changes state that may allow another thread to make progress

Typical monitor code

```
lock
// wait if needed
while (condition) {
    wait
}

do stuff

signal or broadcast about the stuff you did

unlock
```

Administration

- Declare your project group (due Wednesday)
- Work on Project 1

Producer-consumer (bounded buffer)

- Producers fill a shared buffer; consumers empty it
- Need to synchronize actions of producers and consumers

- Why use a shared buffer?
  - Allows producers and consumers to operate somewhat independently (more concurrency)
- Used in many situations
  - Unix pipes (grep “keyword” foo.txt | wc -l)

Coke machine with monitors

- Problem definition
  - Coke machine can hold at most MAX cokes
  - Delivery person (producer) adds one coke to machine
  - Consumer buys one coke
- Step 1: Think about threads independently
- Step 2: Identify shared state
  - State of coke machine: *numCokes*
- Step 3: Assign locks
  - One lock (*cokeLock*) to protect all shared data

Coke machine with monitors

- Step 4: List before-after conditions and assign a condition variable to each condition

Coke machine with monitors

```
int numCokes = 0
```

Consumer()

```
// take coke out of machine
numCokes—
```

Producer()

```
// add coke to machine
numCokes++
```

Reducing number of signals

- Could producer only signal when numCokes changes from 0 -> 1?
- numCokes = 0
- C1 and C2 waiting on waitingConsumers

- P1 increments numCokes to 1 and signals
- P2 increments numCokes to 2, but does not signal

- Only one of C1 and C2 wakes up!

Reducing condition variables

- Can I combine waitingProducers and waitingConsumers into a single condition variable (waitingProdCon)?

Coke machine with monitors

```
int numCokes = 0
mutex cokeLock
cv waitingConsumers, waitingProducers
```

Consumer()

```
cokeLock.lock()

while (numCokes == 0) {
    waitingConsumers.wait(cokeLock)
}


// take coke out of machine
numCokes—

waitingProducers.signal()

cokeLock.unlock()
```

Producer()

```
cokeLock.lock()

while (numCokes == MAX)
    waitingProducers.wait(cokeLock)
}


// add coke to machine
numCokes++

waitingConsumers.signal()

cokeLock.unlock()
```

Reducing condition variables

- Let MAX = 1, and numCokes = 0
- C1 and C2 wait

- P1 increments numCokes to 1 and signals
  - Wakes up C1
- P2 waits because numCokes = MAX

- C1 decrements numCokes to 0 and signals
  - May wake up C2!

- Why is this bad?

Reader-writer locks

- Example: multiple threads would like to read or write Wolverine Access course catalog
- Threads need to acquire lock to read or write shared data

```
Student:
lock()
browse course catalog
unlock()
```

```
Administrator:
lock()
change course catalog
unlock()
```

- How to safely allow more concurrency?

How to safely allow more concurrency?

- Threads disclose whether they intend to read or write the shared data

Student:

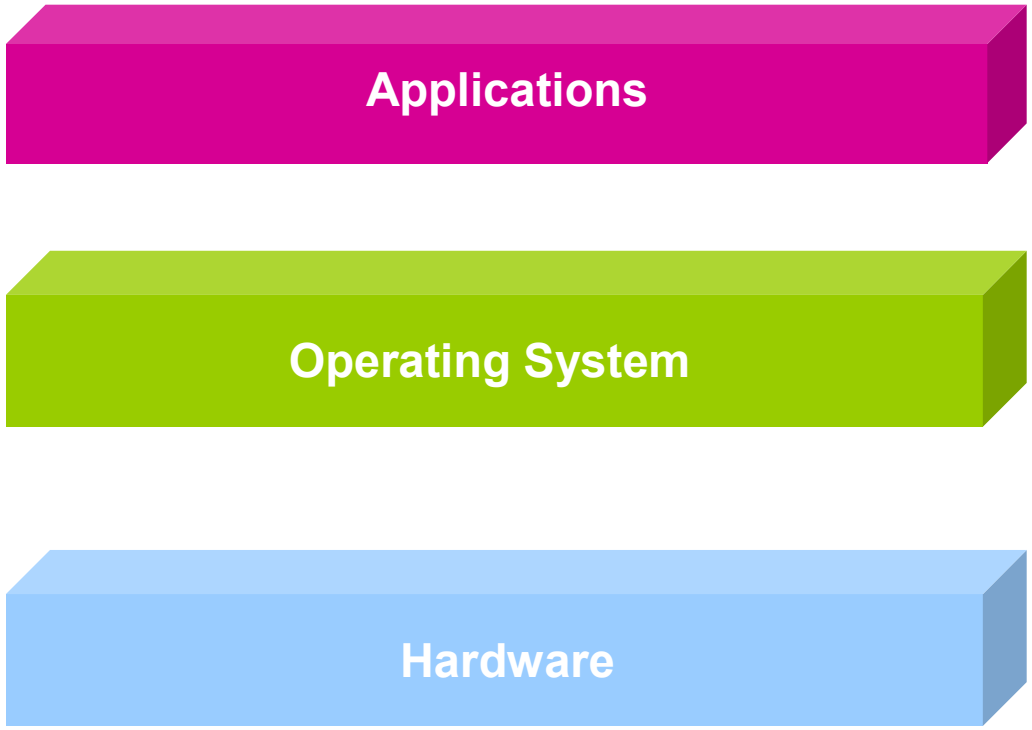
```
lock()  readerLock()
browse course catalog
unlock()  readerUnlock()
```

Administrator:

```
lock()  writerLock()
change course catalog
unlock()  writerUnlock()
```

- Allow multiple concurrent readers, if no threads are writing data
- Allow a single writer, if no other threads are reading or writing

Another level of abstraction



Even higher-level synchronization operations
Concurrent programs
(readerLock, readerUnlock, writerLock, writerUnlock)

Higher-level synchronization operations
(lock, monitor, semaphore)

Atomic operations
(load/store, interrupt enable/ disable, test&set)